

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science And Business Analytics"

UDC 004.89,005,311.21,69

Research Team Project Report on the Topic:
NLP-based Ethics Index for Housing Developers
(interim, the first stage)

Submitted by the Students:

group #БПАД221, 3rd year of study

Budyak Kirill Aleksandrovich

group #БПАД222, 3th year of study

Korableva Viktoriia Yurievna

Approved by the Project Supervisor:

Storchevoi Maksim Anatolyevich

Research Fellow

Faculty of Computer Science, HSE University

Contents

Annotation	4
Keywords	4
1 Introduction	5
1.1 Understanding ESG	5
1.2 ESG in the real estate sector	5
1.3 Research goals	6
1.4 Unique contributions and relevance	7
1.5 Structure of the report	7
1.6 Job Distribution	7
2 Literature review	8
3 Data Collection	8
3.1 Selection of Developers	9
3.2 Data Sources: Avito and CIAN	9
3.3 Web scraping Strategy and tools	10
3.4 Custom parser implementation	12
4 Data Preprocessing	15
4.1 Merging Text Fields	15
4.2 Filtering Incomplete or Invalid Entries	16
4.3 Normalization and Cleaning of Fields	17
5 BERT-based Sentiment Classification Pipeline	20
5.1 Introduction to BERT for Sentiment Analysis	20
5.2 Data Preparation and Label Definition	21
5.3 Text Pre-processing	21
5.4 Tokenisation and Dataset Preparation	22
5.5 Model Architecture and Training Setup	23
5.6 Hyper-parameter Tuning	25
5.7 Final Model Training and Evaluation	26

6	Clustering Analysis	28
6.1	Data Integration and Feature Engineering	28
6.2	K-Means Clustering: Methodology and Rationale	29
6.3	HDBSCAN Clustering: Methodology and Comparison	29
7	Topic Extraction with TF-IDF and BERT Embeddings	31
7.1	Methodology: c-TF-IDF for Cluster Keywords	31
7.2	Topics obtained	31
8	S-Index Calculation (Developer Trust Metric)	32
8.1	3.1 Defining the S-Index and Its Relevance	32
8.2	3.2 Approaches Tested for S-Index Computation	33
8.3	3.3 Evaluation of All S-Index Variants	35
8.4	3.4 Rescaling to 0–5 and Top-10 Leader Board	35
8.5	3.5 Pre-processing Pipeline for the Logistic Model	35
8.6	3.6 Summary	36
9	Conclusion	36
	References	39

Annotation

This project aims to assess the social responsibility of real estate developers by analyzing unstructured customer reviews from specialized platforms. The system integrates a custom parser for data collection with Natural Language Processing (NLP) techniques, including a fine-tuned BERT model for sentiment analysis and TF-IDF-based clustering to identify key social responsibility themes. By dynamically weighting categories such as client interaction and labor conditions, the index provides a normalized score (0–100) for objective comparison. The methodology emphasizes transparency, scalability and adaptability to evolving social priorities in the construction sector.

Аннотация

Данный проект разрабатывает систему оценки социальной ответственности девелоперов на основе анализа отзывов с платформ, специализированных на секторе недвижимости. Система включает парсер для сбора данных, модель BERT для анализа тональности и кластеризацию TF-IDF для выявления социальных тем. Индекс использует динамические веса категорий (взаимодействие с клиентами, условия труда) и обеспечивает прозрачность расчетов. Методология ориентирована на масштабируемость и адаптацию к изменяющимся социальным приоритетам.

Keywords

ESG index, social responsibility, Russian real estate developers, data collection, data cleaning, BERT fine-tuning, sentiment analysis, NLP, TF-IDF clustering, data parsing, ethical scoring

1 Introduction

1.1 Understanding ESG

Environmental, Social, and Governance (ESG) principles are essential in assessing the sustainability and ethical responsibility of businesses. ESG metrics have become a central focus in investment decisions and corporate strategies, influencing how companies are evaluated by investors, regulators, and consumers.

- **Environmental (E):** assesses a company's impact on nature, including carbon emissions, energy efficiency, and resource usage.
- **Social (S):** focuses on a company's relationships with employees, customers, and communities, covering aspects such as labor conditions, diversity and inclusion and customer satisfaction
- **Governance (G):** evaluates corporate transparency and diversity, anti-corruption measures and ethical business practices

1.2 ESG in the real estate sector

The real estate industry plays a critical role in ESG, particularly in its social and environmental impact. Developers are judged based on their sustainability efforts, compliance with labor laws, and broader influence on urban development. While environmental and governance aspects are well-documented, the Social dimension remains underexplored due to the difficulty in assessing aspects like customer sentiment which is subjective and constantly changing and, also, stakeholders trust.

Existing ESG ratings for real estate developers are primarily based on company-disclosed data, sustainability reports and third-party expert evaluations. These traditional approaches often lack real-time insights, making them vulnerable to biases and limited in their ability to reflect actual stakeholder concerns. Therefore, there is an increasing demand for more data-driven, user-centric methodologies that reflect real-world perceptions of developers' social responsibility.

1.3 Research goals

This project introduces a novel methodology for constructing an ESG social responsibility index specifically for real estate developers. By leveraging sentiment analysis of user-generated reviews and discussions on online platforms, our approach extracts unbiased, real-time insights into public opinion. Natural language processing (NLP) techniques, combined with machine learning models, allow for an automated and scalable method of assessing social responsibility metrics. The core objectives of this project are:

- **Data collection and preprocessing:** creating a list of development companies, aggregating user reviews from various online sources such as real estate forums, rating platforms, and social media using web-scraping technologies
- **Sentiment classification:** implementing a fine-tuned BERT model to classify sentiment (positive, negative, neutral) within extracted reviews.
- **Topic clustering:** applying TF-IDF and clustering techniques to detect key discussion themes relevant to social responsibility.
- **Index computation:** formulating a weighted scoring system to measure social responsibility of developers across categories , such as customer service, housing quality, environmental impact, and labor conditions.
- **Visualization and interpretation:** presenting results in an interactive dashboard, allowing users to explore social responsibility trends dynamically.

1.4 Unique contributions and relevance

This research addresses a critical gap in ESG assessments by integrating real-time public sentiment into developer evaluations. Unlike traditional ESG indices that depend on structured financial data and subjective expert evaluations, our model ensures a data-driven, unbiased and adaptive approach to social responsibility assessment. The ability to dynamically detect emerging themes and adjust weightings based on evolving public concerns makes this system particularly resilient and scalable compared to existing methodologies.

Moreover, the real estate sector lacks standardized approaches to quantifying social responsibility using user-generated content. Our model provides an alternative perspective, enhancing ESG evaluation transparency while offering valuable insights to investors, customers and regulatory bodies.

1.5 Structure of the report

The rest of this report is organized as follows:

- **Literature review:** analysis of existing ESG assessment frameworks and their limitations.
- **Methodology:** a breakdown of data sources, NLP techniques, sentiment classification, and index computation.
- **Implementation:** detailed explanation of model training, clustering, and visualization.
- **Results and discussion:** Presentation of findings, performance evaluation, and practical implications.
- **Conclusion:** Summary of key contributions, challenges, and future research directions.

1.6 Job Distribution

Korableva Viktoriia: Collection of the relevant sites with reviews, tuning parsers for review collection from Avito, validating BERT model, working with TF-IDF and k-means/HDBSCAN to detect key discussion themes relevant to social responsibility, Index computation.

Budyak Kirill: Collection of the relevant companies, parser deployment for CIAN, working with BERT model to classify sentiment (positive, negative, neutral) within extracted reviews, Index validation.

2 Literature review

In the field of NLP for ESG analysis, several models aim to automate the classification and evaluation of textual data. However, most existing solutions are either focused on corporate reports or news articles, making them less relevant to our task—assessing the social responsibility of real estate developers based on user reviews.

One of the most well-known tools is Sberbank ESGify [15], which utilizes a BERT-based model to classify ESG risks based on corporate documents and legal texts. However, its application is limited to analyzing formalized sources of information, whereas our work focuses on user-generated content, which better reflects the real perception of developers’ social responsibility

The study by Pontes et al. (2023) [14] explores multilingual ESG issue classification (English, Chinese, French) using the RoBERTa model. The main limitation of this approach is its lack of specialization in the real estate sector and its inability to handle Russian-language data. In contrast, our model is specifically designed to analyze developer reviews in the Russian market.

The work of Mehra et al. (2022) [16] introduces ESGBERT, a BERT-based model that predicts changes in companies’ environmental scores based on corporate reports. While this approach effectively identifies ESG score variations, it does not fit our task, as it focuses solely on financial reports and neglects consumer opinions.

Lastly, Chen et al. (2023) [3] presents a multilingual classification system for ESG-related news articles, following MSCI ESG rating principles. Despite its advanced categorization mechanism, this method is not applicable for assessing developers’ social responsibility, as it does not analyze user reviews and is not tailored to the specifics of the Russian real estate market.

Thus, our research fills a critical gap in ESG evaluation by:

- Specializing in real estate, unlike general ESG solutions
- Using user-generated reviews instead of corporate documents
- Employing advanced NLP techniques, including fine-tuned BERT and topic clustering
- Adapting to Russian-language data, making it unique in the context of ESG analysis for the Russian real estate market

3 Data Collection

In order to evaluate the *social responsibility* (“S-score”) of real estate developers using online user reviews, we first built a comprehensive dataset of such reviews from two major Russian real estate platforms: **Avito** and **CIAN**. These platforms were chosen as they host large numbers of

user reviews on residential properties and developers, providing rich, crowd-sourced information about customer experiences. This section describes how we selected the developers of interest, the sources and methods of data collection (including the development of custom web parsers) and the structure of the data gathered.

3.1 Selection of Developers

Not all developers have an equal online presence or volume of feedback on Avito and CIAN. To ensure our analysis is meaningful, we focused on a curated list of **38 major real estate developers in Russia**. These developers were selected based on three primary criteria: (1) their **head offices** are in Moscow/Moscow Region or in Saint-Petersburg/Saint-Petersburg region, (2) their broad **coverage on both platforms** - they actively list properties or are discussed on both Avito and CIAN and (2) the **quantity of available user reviews** associated with them. In practice, we surveyed the developers featured on Avito and CIAN and chose those who had a adequate number of reviews (to guarantee statistically significant insights) and were present on both platforms (to maximize coverage of user opinions). By filtering in this way, we included developers that are both active in the market and have generated considerable customer feedback online. This avoids cases where a developer might be large but has little online feedback, or conversely, has some reviews but only on one platform. The result was a balanced set of 38 developers, ensuring that the subsequent analysis of their S-scores would be grounded in sample data from multiple sources. Focusing on this set allows us to capture diverse user sentiments and issues, from construction quality and delays to customer service and community impact, as reflected in the reviews. In total, across these developers, we collected **18048 reviews** spanning approximately the years 2020 through early 2025. This multi-year range provides a longitudinal perspective on each developer’s performance and reputation.

3.2 Data Sources: Avito and CIAN

Our data comes from two key online platforms in the Russian real estate market:

- **CIAN** – a popular real estate portal that includes dedicated sections for *residential complexes* and their developers. CIAN allows users to leave reviews and ratings for specific residential projects, which often implicitly reflect on the developer responsible for those projects. It serves as a rich source of feedback on new buildings, covering aspects such as construction quality, delays in delivery, infrastructure and the developer’s reliability.
- **Avito** – Russia’s largest online marketplace, which also has a real estate section. In recent years,

Avito introduced the ability for users to rate and review sellers, including professional real estate developers or agencies on their platform. Many developers maintain official seller accounts on Avito to list apartments in their new projects. Customers who purchased or interacted regarding those listings can leave a review on the developer’s profile (or on the specific property listing). These reviews on Avito provide direct consumer feedback about the developer, complementing the project-specific insights from CIAN.

By drawing data from both CIAN and Avito, we capture two complementary angles: project-level opinions (CIAN’s reviews of individual residential complexes, which reflect on the developer’s work in that project) and developer-level reputation (Avito’s reviews of the developer as a seller or company) - combining these factors give us a full review on the industry. Each platform’s reviews were accessed in their respective sections dedicated to user feedback. For example, on CIAN we navigated to pages listing reviews for a given residential complex or developer, and on Avito we accessed the developer’s profile page where buyer reviews were listed. Both sources typically provide a star rating and free-text comments from users, along with metadata like dates and usernames, which we also collected.

3.3 Web scraping Strategy and tools

To gather the reviews at scale, we developed a **custom web scraping script** (parser) for each platform. The goal was to automate data collection such that for each of the 38 selected developers, all available reviews on both Avito and CIAN would be retrieved and stored. The scraping process faced two main technical challenges: *dynamic content loading* and *anti-scraping measures*. The review sections on these websites are not simple static HTML – they often load additional reviews dynamically (e.g., upon scrolling or clicking “show more”) and the sites may employ measures to detect and block automated bots. We addressed these challenges by carefully choosing our scraping tools and approach, as described below.

Initial approach with requests and BeautifulSoup:

We first considered using a traditional scraping approach: Python’s **requests** library to fetch HTML content and **BeautifulSoup** for parsing the page. *BeautifulSoup* is a robust HTML parsing library that works well for extracting data from static HTML pages. We planned to identify the HTML elements corresponding to review content (e.g., review text, username, date, etc.) by inspecting the site’s structure, and then use BeautifulSoup to parse and extract those fields.

However, during testing this approach on CIAN and Avito, we encountered significant difficul-

ties:

- **Dynamic content not retrieved:** many reviews were not present in the initial HTML that `requests` returned. Both Avito and CIAN load additional reviews or details via JavaScript after the initial page load. For example, a page might initially show only a few reviews, with the rest fetched on-the-fly as the user scrolls or clicks a pagination button. Because `requests` (and BeautifulSoup by extension) only sees the raw HTML returned from the server (without executing any JavaScript), it often missed a large portion of the reviews.
- **Anti-Bot mechanisms:** the platforms sometimes presented bot-detection challenges (like requiring human interaction or blocking rapid successive requests). Using a simple requests-based scraper, even when we tried tactics like adding delays and spoofing the “User-Agent” string (to pretend to be a normal web browser), led to frequent blocks or incomplete data. For instance, after a few page requests, the server might stop responding with data or present a captcha. This made pure HTTP scraping unreliable.

Given these issues, it became clear that a more robust solution was needed to scrape dynamic, protected content. Instead of trying to manually reverse-engineer the platforms’ API calls or bypass their anti-scraping checks (which could be time-consuming), we opted for a browser automation approach.

Adopting Playwright for Dynamic Scraping: We turned to *Playwright*, a modern browser automation library, to drive the data collection. Playwright allows us to launch a real browser (headless or headful Chrome/Chromium, Firefox, etc.) and programmatically interact with web pages, simulating a human user. By using Playwright with a headless Chromium engine, we effectively let the site load exactly as it would in a normal web browser, including executing all JavaScript and rendering dynamic content such as loaded-on-scroll reviews. This approach had several key advantages over the simple requests/BeautifulSoup method:

- **Dynamic content extraction:** Playwright can wait for and retrieve content that only appears after certain user actions or time delays. For example, for a CIAN residential complex page that initially shows 10 reviews with a “Show more” button, our Playwright script can click that button (or scroll to trigger auto-load) and wait for the next batch of reviews to render, continuing until all available reviews are loaded. This ensured no reviews were left uncollected due to dynamic loading.
- **Reduced anti-bot friction:** because Playwright uses a real browser context, the requests it makes include all the typical browser headers and behavior by default. This meant we did not need to manually spoof a User-Agent or manage cookies – the browser context handled that.

The result was **fewer anti-bot problems**: the platforms treated our scraper essentially like a normal user’s browser. We observed far fewer blocks or challenges, and could even automate login or other interactions if necessary (though for reading public reviews, login was not required on these sites).

- **Complete page rendering:** using a headless Chromium instance gave us a fully rendered Document Object Model (DOM) to work with. We could use Playwright’s page querying functions (similar to how one might use JavaScript in the browser) or even export the final HTML after all content loads and parse it with BeautifulSoup. In practice, we often found it convenient to use Playwright’s selectors to directly extract text. For example, we identified CSS selectors or XPath for elements like the review text, rating stars, date, etc., and let Playwright fetch their inner text values once they appeared on the page.
- **Automation of interactions:** Playwright allowed us to script necessary interactions – such as clicking pagination links, pressing a “load more reviews” button, or scrolling down incrementally – to ensure all content was gathered. This would have been cumbersome to replicate with requests alone (one would have to manually find and call the platform’s internal APIs or endpoints for loading more reviews, which could change or be protected).

In summary, while **BeautifulSoup** is an excellent tool for parsing HTML, it relies on having the complete HTML content, which we could not reliably obtain via direct requests due to dynamic loading and anti-scraping measures. **Playwright**, on the other hand, provided a robust, if heavier, solution by running a real browser. The overhead of launching a browser for scraping was justified by the completeness and reliability of the data extraction. We did not encounter the need for explicit user-agent spoofing or elaborate workarounds; the Chromium engine’s default behavior sufficed to mimic a typical user. The use of Playwright significantly reduced the chances of being blocked or served incomplete content. Once the pages were fully loaded and all reviews visible, we either extracted the data via DOM selectors or optionally pulled the page source into BeautifulSoup for final parsing.

3.4 Custom parser implementation

Using Playwright, we implemented custom parsers for each site, tailored to their HTML structure. In essence, for each developer in our list:

- For **CIAN**: the parser navigated to the section of the site where user reviews for a given residential complex or developer are listed. In CIAN’s interface, users often leave reviews on specific *residential complex pages*. We leveraged the fact that we know which projects (residential

complexes) are associated with our target developers. For each such project, the script opened its reviews page, then iteratively scrolled or clicked to load all reviews. We then extracted each review’s content. If CIAN also provides an aggregated developer-level review page, those were similarly scraped. Key data points like the reviewer’s name, the date, the star rating, and the pros/cons text were captured using DOM element selectors identified through inspecting the site’s HTML structure (e.g., class names or element attributes that denote review text).

- For **Avito**: the parser accessed the developer’s profile or seller page where their customer reviews are displayed. On Avito, a developer with multiple listings is treated somewhat like a “store”, and customers can leave feedback on that store. The script opened the developer’s review section and scrolled through or paginated until all reviews were visible. We then extracted the needed fields. In Avito’s case, we observed that the reviews also contained star ratings, user names, dates, and comment text. Avito’s dynamic content (if any) was handled in the same way by letting the page fully render. Avito occasionally uses infinite scroll for loading more reviews, which our script handled by programmatically scrolling down in steps until no new reviews loaded.

During parsing, we took care to **throttle the scraping speed** to some extent, inserting a small delay between interactions, to further reduce the load on the servers and avoid triggering anti-scraping alarms. For instance, after loading a batch of reviews, the script might wait a couple of seconds before attempting to load more, simulating a real user reading time. This polite crawling strategy ensured we gathered data without excessively hammering the websites. All extracted information for each review was stored in an in-memory data structure (e.g., a Python dictionary or list of records) and periodically written out to a structured file. We used a CSV (Comma-Separated Values) format to store the collected data, as it is easy to handle for subsequent analysis. Each row in the CSV corresponds to a single user review.

Field	Description
<i>Developer</i>	The name of the real-estate developer to which the review pertains (one of the 38 companies in our sample). This field allows grouping of reviews by developer.
<i>User</i>	The user name (nickname) of the reviewer as displayed on the platform. While not directly analysed, it helps verify review diversity.
<i>Date</i>	Calendar date on which the review was posted (captured as <i>dd-mm-yyyy</i>). It allows us to study trends over time.
<i>Rating</i>	Numerical star rating (1–5, half-stars possible). Serves as a quantitative sentiment indicator complementing the text.
<i>Project name</i>	Name of the residential complex (<i>ZhK</i>) when the review targets a specific project (field <i>zhk_name</i>). Empty when the comment is general to the developer.
<i>Summary</i>	One-line headline provided by the author, e.g. “Good value for money”. Captured when available because it often condenses the overall sentiment.
<i>Advantages</i>	Positive aspects mentioned by the user, such as “Close to schools” or “Flexible payment plan”. Stored verbatim for downstream topic analysis.
<i>Disadvantages</i>	Negative aspects, e.g. “Poor sound insulation” or “Delay in handing over the keys”. Key for identifying social-responsibility issues.

Table 3.1: Key data fields extracted from user reviews on Avito and CIAN for each developer.

In addition to the fields listed in Table 3.1, we also recorded metadata such as the source platform of the review (Avito vs. CIAN) when merging data, and the URL or unique ID of the review where available (for traceability). However, the primary fields of interest remain those capturing the content of the review and its basic context (who, when, what rating, and which project/developer). All the collected data was stored in a consolidated CSV file, ensuring that each review from either platform is represented as one row with all the above fields.

Each of the 38 developers has dozens to hundreds of reviews in the dataset and finally we assembled 18 048 reviews overall. This raw dataset is a backbone for the subsequent preprocessing and analysis steps. In the next section, we detail how we cleaned and preprocessed this data to prepare it for sentiment analysis and topic modeling.

4 Data Preprocessing

Having collected a substantial amount of raw review data from Avito and CIAN, the next step was to preprocess and clean this data before applying analytical models. Raw user-contributed data can be messy or inconsistent – it may contain missing values, irregular formatting, or other issues that could interfere with analysis. Our preprocessing focused on transforming the dataset into a structured, consistent, and information-rich form suitable for techniques like sentiment analysis (with BERT) and topic extraction (with C-TF-IDF and clustering) later in the project. In this section, we describe the key preprocessing steps: merging text fields for analysis, filtering out incomplete entries, and normalizing both textual and numeric fields.

4.1 Merging Text Fields

Each review in our dataset initially had up to three separate text fields describing the user’s opinion: *summary*, *pros* and *cons*. While having these separated is useful for certain analyses (for example, one could specifically analyze all “pros” to see what positive aspects are most often mentioned), many natural language processing tasks (such as sentiment classification or topic modeling) work on a per-document basis, where each document is a single block of text. For our purposes, it was advantageous to **combine the multiple text fields of each review into one text** that represents the full content and overall sentiment of that review.

Method: for each review, we created a new text field which is essentially a concatenation of the summary, advantages and disadvantages. We took care to include appropriate separators or spacing so that the sentences from different fields don’t run together unintelligibly. For example, if a review has a summary, we might start with that as an introductory sentence, then perhaps label the pros and cons or at least separate them by punctuation.

If a particular field was missing (e.g., some reviews did not have a distinct summary or perhaps a reviewer listed only pros but no cons), our concatenation logic would simply omit that part. We did *not* introduce any artificial text if a field was empty; we just skipped it or, in some cases, used an empty string for it. The key point is that every review ends up with a single textual representation that includes whatever commentary the user provided.

By doing this merging, we facilitated later stages where each review could be treated as one document with a cohesive text. This is especially helpful for our **BERT-based sentiment analysis**, which processes each review as a single sequence of text, and for **topic modeling**, which typically treats each document as an independent text sample from which to extract common themes.

It’s important to note that the original structured form (separate pros, cons, etc.) was not lost – we preserved those in the data as well. But for modeling purposes, we often used the merged text. Keeping both forms allows flexibility: one can always refer back to structured fields if needed (for example, to specifically report what proportion of reviews mention a certain advantage), even while the models might use the merged form for training or inference.

4.2 Filtering Incomplete or Invalid Entries

After merging text fields, we examined the dataset for any **incomplete, empty, or invalid entries** that should be removed or corrected. Data filtering is a crucial step to ensure that only meaningful and analyzable reviews remain. Several scenarios were considered:

- **Empty text:** reviews that ended up with no text content at all after merging fields. This could happen if, for instance, a user left only a star rating without any written comment, or if all text fields were missing/blank. Such entries are not useful for textual analysis (since there’s no text to analyze for sentiment or topics), and they were dropped from the dataset. In practice, we found a small number of cases where a review contained a rating but the user did not fill in any of the text fields (no summary, no pros, no cons). We removed these entries.
- **Missing rating or other key fields:** on the other hand, a review with text but missing a rating could still be used for certain analyses (like topic modeling), but it would be problematic

for analyzing sentiment if we wanted to correlate with star ratings. We decided to retain reviews that have text even if the star rating is missing, since the text can still be processed for sentiment via BERT (which doesn't require the star rating). However, we flagged such cases. Fortunately, the platforms generally require a star rating when leaving a review, so virtually all reviews had a rating.

- **Duplicate entries:** we checked if any reviews were unintentionally collected twice (for example, if a developer's project reviews and developer profile reviews overlapped or if our scraper accidentally repeated entries). Using unique identifiers or a combination of fields (developer name + user + date + a snippet of text), we did not find significant duplication. In cases where duplicates were found, we would remove them to avoid double-counting the same opinion.
- **Out-of-scope content:** We also skimmed through the reviews to ensure they were indeed relevant to evaluating the developer. Since our scraping targeted specific sections, nearly all data was on-topic. We did not have issues like advertisements or irrelevant forum posts because of the controlled way we collected data. Thus, no major content-based filtering was needed beyond empty entries.

After this filtering step, we had a "clean" set of user reviews. Each remaining review had at least some textual content (summary/pros/cons) and a rating. This resulted in a slight reduction in the total number of reviews (due mostly to dropping of empty ones), but ensured that every review we analyze going forward contains substantive information.

4.3 Normalization and Cleaning of Fields

With the dataset filtered and all reviews merged into single texts, we performed various **normalization and cleaning operations** on the fields to standardize the data format and correct any minor issues. This included:

Text Cleaning: the combined review texts (as well as the original pros/cons fields) were cleaned to remove any artifacts from scraping and to normalize the content. This involved steps such as:

- Removing or converting any HTML entities or markup that may have been captured. In some cases, special characters like ampersands (&) or quotes might appear as HTML codes (e.g., """ for a quote) in the raw data. We converted those to the proper characters.
- Stripping excessive whitespace, newlines, or tab characters. For example, if a user's pros or cons section had line breaks or if the concatenation introduced double spaces, we cleaned that so that the text is a neat, continuous narrative. We generally kept sentence punctuation (periods,

commas) as-is, but ensured spacing was standardized (single space between words/sentences).

- Preserving language-specific characters: Since many reviews are in Russian, we ensured that our text encoding (UTF-8) was preserved and all Cyrillic characters are intact. Our cleaning did not involve removing or altering non-Latin characters—on the contrary, we specifically maintained them so that the Russian text is correctly processed by our models (which are capable of handling Cyrillic).
- Lowercasing: for certain analyses like C-TF-IDF, it is common to convert text to lowercase to avoid treating “Developer” and “developer” as separate words, for instance. We created a lowercased version of the text for any model or analysis that would require case insensitivity. However, for BERT-based sentiment analysis, which is case-sensitive and where the pre-trained model expects cased text, we left the text in its original casing. Thus, our pipeline was flexible: we could use either the original-cased text or the lowercased text depending on the requirements of the analysis step.
- No heavy normalization like stemming at this stage: we did *not* perform stemming or lemmatization in the initial cleaning stage. The reasoning is that we wanted to preserve the natural form of the text for the BERT sentiment model (which can understand inflections in context) and also because we planned to handle any needed linguistic normalization later during the topic modeling phase. For example, when using TF-IDF on Russian text, one might apply lemmatization to group words by root (so that “строительство” and “строитель” are recognized as related to “строить”, i.e., building). We deferred such linguistic preprocessing to the stage where it’s necessary (and we will describe that in the topic extraction section of the report). In the initial cleaning, our focus was on structure and basic consistency rather than deep linguistic processing.

Numeric Rating Normalization

We also standardized the format of the **rating** field:

- All rating values were converted to a uniform numeric scale (1 to 5). Both Avito and CIAN use five-star rating systems, so in theory they were already on the same scale. We ensured to store them as numeric (integer or float) types rather than strings. In the raw data, some ratings came as "5" (a string), others might have been "4.0" or "4,5" (using a comma or period for half-stars). We parsed these consistently. For instance, a “4,5” (if comma was used as decimal in the Russian locale) was interpreted as 4.5.
- We decided how to handle half-star ratings: if a platform allowed half increments (like 4.5), we kept those as is (floating point). If we needed an integer for certain analyses, we could round or floor them, but we generally preserved the precision at this stage.

- We also double-checked that no ratings fell outside the expected range (sometimes a scraping error could capture an unrelated number). All values were indeed between 1 and 5, so that was consistent.
- If any reviews had a missing rating (which was rare), we left that field blank or null. As noted, such reviews were few and could be omitted from analyses that specifically need ratings (like calculating an average score), or we could impute a neutral rating if absolutely needed. However, since essentially all reviews had ratings, this wasn't a significant issue.

Date Formatting:

The **date** of each review was normalized to a standard format to enable chronological analyses if needed:

- On CIAN and Avito, dates were typically displayed in a human-readable format, e.g., “13 декабря 2024” (in Russian) or sometimes relative dates like “2 months ago”. Our parser captured the exact strings. We converted month names from Russian to an international numeric format (e.g., “декабря” to 12 for December) and reformatted the dates as ISO 8601 (YYYY-MM-DD) for consistency. In the example above, “13 декабря 2024” became 2024-12-13.
- For relative dates (if any), we translated them to absolute dates. For instance, if a review was scraped in March 2025 and it said “2 месяца назад” (“2 months ago”), we would calculate the date (approximately January 2025). However, most reviews on these sites show actual dates once they are a certain age, so this was not a common issue.

Other Field Normalizations:

We also ensured consistency in other categorical fields:

- The **developer names** were normalized so that the same developer always has the exact same spelling in the dataset. For example, if one source called a developer “Company ABC” and another “ABC, Inc.”, we unified them under one name (since we already know the 38 developers we are focusing on, we used those official names as keys). This way, grouping by developer is straightforward.
- The **project names** were cleaned for consistency (removing any extra quotes or spacing). No further transformation was done on them beyond what was needed to ensure that if the same project was mentioned in multiple reviews, it was spelled identically every time.
- The **usernames** were left mostly as-is, since they don't impact analysis except possibly to identify duplicates. We only removed any leading/trailing spaces or newline characters from usernames. We did not attempt to anonymize or encode them, since they are usually first names or nicknames and already somewhat anonymized by the platforms.

With these preprocessing steps, the dataset was transformed into a clean and structured

form. Each review now is represented by a single textual field containing the full review content (merged summary/pros/cons), accompanied by normalized metadata (developer, project, date, rating, etc.)

The result of preprocessing is a dataset that is ready for analysis. In the subsequent sections, we will leverage this prepared data for various analytical tasks: using a BERT-based model to assess the sentiment of each review, applying C-TF-IDF and clustering techniques to extract key discussion topics from the reviews, and finally constructing an S-score index to quantify each developer’s social responsibility performance based on these findings.

5 BERT-based Sentiment Classification Pipeline

5.1 Introduction to BERT for Sentiment Analysis

Bidirectional Encoder Representations from Transformers (BERT) [5] is a state-of-the-art, transformer-based language model that can be fine-tuned for sentiment classification tasks. BERT employs a sub-word tokenizer that splits an input sentence into a sequence of tokens—including two special markers required by the model [12]:

[CLS] the apartment is beautiful [SEP]

Here [CLS] is a *classification token* automatically added at the beginning of every sequence, and [SEP] marks its end. The resulting token IDs are passed through several self-attention layers, yielding contextualised embeddings $\mathbf{h}_t$ for every position t . The final hidden state corresponding to [CLS], denoted $\mathbf{h}_{[\text{CLS}]}$, is treated as an aggregated representation of the entire sentence.

A lightweight classifier head—typically a single linear layer—maps this pooled embedding to a vector of class logits $\mathbf{z} \in \mathbb{R}^C$, where C is the number of sentiment classes (negative, neutral, positive in our case). The prediction probabilities are obtained via the soft-max transformation

$$P(y = c \mid \mathbf{x}) = \text{softmax}(\mathbf{W} \mathbf{h}_{[\text{CLS}]} + \mathbf{b})_c, \quad c = 1, \dots, C.$$

The model outputs the class with the highest probability as its sentiment prediction. For illustration, suppose the network returns $[0.33, 0.41, 0.26]$ for {negative, neutral, positive}. Because the neutral class attains the highest likelihood (0.41), the review “*The apartment is beautiful*” would be classified as **neutral**. This toy example shows how the [CLS] representation, together with a linear classifier, enables BERT to convert raw text into a sentiment label.

5.2 Data Preparation and Label Definition

Our sentiment corpus consists of Russian-language customer reviews about residential developers collected from Avito and CIAN. Each review carries a *five-star* user rating (1 = worst, 5 = best). To obtain categorical sentiment targets for supervised training we collapsed the five-point scale into three intuitive classes:

Numeric rating	Assigned sentiment label
1–2 stars	0 = Negative (strong dissatisfaction)
3 stars	1 = Neutral (mixed / average sentiment)
4–5 stars	2 = Positive (clear satisfaction)

Illustrations.

- A one-star complaint about persistent construction delays $\rightarrow$ *Negative (0)*.
- A three-star remark balancing pros and cons $\rightarrow$ *Neutral (1)*.
- A five-star praise for high build quality $\rightarrow$ *Positive (2)*.

After labelling, the full dataset comprised 18048 reviews.

Class frequencies were *imbalanced*—a common pattern in real-world data:

Sentiment	Count
Negative (0)	2246
Neutral (1)	1307
Positive (2)	14495

The neutral class is the smallest because relatively few users assign an exactly three-star score, whereas many give strong approval or disapproval (1–2 or 4–5 stars). This skew informed subsequent modelling choices—particularly the use of class weighting and sampling strategies. The resulting labels (0, 1, 2) and their distribution constitute the ground truth for training and evaluating the BERT classifier.

5.3 Text Pre-processing

Prior to tokenisation with RuBERT, we applied a dedicated clean-up and normalisation pipeline to every Russian review. All steps were implemented with the open-source **Natasha** toolkit—an NLP library tailored specifically for Russian morphology.

1 Tokenisation and Sentence Segmentation

Natasha first splits each review into sentences and word-level tokens (punctuation included). This isolates linguistically meaningful units and discards stray technical characters.

2 Lemmatisation

Russian is highly inflected: the noun *застройщик* (“developer”) appears as *застройщика*, *застройщику*, etc. Natasha’s morphological analyser converts every token to its dictionary form (*lemma*). Hence *квартиры* and *квартира* both become *квартира* (“apartment”). Lemmatisation reduces vocabulary size, groups synonymous inflections, and markedly lowers data sparsity—crucial for Russian sentiment modelling [7].

3 Lower-casing and Noise Removal

Because our BERT backbone is cased-agnostic, all text is converted to lower case. We then strip or normalise non-linguistic symbols (HTML remnants, emojis, excess punctuation, elongated letter sequences, stray numbers). Common function words are *retained*: BERT learns their limited importance automatically, and wholesale stop-word removal is unnecessary for transformer models.

These cleaning operations remove noise while respecting Russian morphology. By collapsing inflected variants into a single lemma the effective vocabulary shrinks, enabling BERT to focus on sentiment instead of memorising orthographic alternations (e.g., *плохой*, *плохого*, *плохому* - all map to one token *плохой*, “bad”). Also, this normalisation yields better generalisation and higher evaluation scores. After pre-processing, the cleaned reviews proceed to BERT tokenisation and model training.

5.4 Tokenisation and Dataset Preparation

All cleaned texts were converted into the numerical format expected by BERT with the `ruBERT-large tokenizer`^{1,2}. The tokenizer employs *byte-pair encoding* (BPE) to segment Russian words into sub-word units, which preserves common prefixes and suffixes and gracefully handles out-of-vocabulary terms.

- 1 **Special tokens:** a classification token [CLS] is prepended to every review; [SEP] marks the end of the sequence.
- 2 **Sub-word segmentation:** Each Russian word is split into one or more BPE pieces. Rare or compound tokens are decomposed (sub-sequent fragments are prefixed “##”), enabling the fixed vocabulary to represent virtually any word form.

¹Model checkpoint `ai-forever/ruBert-large` on [HuggingFace](#).

²Architecturally equivalent to English BERT-large, but pre-trained on a multi-billion-token Russian corpus.

- 3 **ID mapping:** every piece is translated to its integer ID in the `ruBERT` vocabulary, producing the input ID sequence $\langle t_1, \dots, t_n \rangle$.
- 4 **Length management:** sequences longer than the model limit (≤ 512 tokens) are truncated. In practice, fewer than 1% of reviews exceeded this length.
- 5 **Auxiliary masks:** an *attention mask* distinguishes real tokens from padding; token-type IDs are set to 0 (single-sentence inputs).

Dynamic padding: instead of padding every sample to the global maximum of 512 tokens—which wastes computation for the many short reviews—we employed `DataCollatorWithPadding`. At run-time it pads each mini-batch only to the length of the *longest* sequence in that batch (e.g. 128 tokens), substantially reducing GPU memory usage and training time while retaining efficient parallelism.

Train–test split and stratification: after tokenisation we performed a stratified split, holding out **20 %** of the data as an unseen test set. Stratification preserved the original class proportions (negative : neutral : positive), which is critical given the class imbalance (neutral reviews were markedly fewer). The remaining 80 % constituted the *training corpus*. All hyper-parameter search and cross-validation were carried out exclusively on this training portion, again using stratified folds to ensure reliable performance estimates across the minority class.

5.5 Model Architecture and Training Setup

Base model: we fine-tuned `BertForSequenceClassification` built on the *ruBERT-large* encoder (24 transformer layers, hidden size 1024). The network produces the pooled $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{1024}$ vector for the [CLS] token, which is passed through

- a dropout layer ($p=0.1$ by default) to mitigate co-adaptation, and
- a single fully-connected layer $\mathbf{W} \in \mathbb{R}^{3 \times 1024}$ yielding the logit vector $\mathbf{z} = \mathbf{W} \mathbf{h}_{[\text{CLS}]}$, one logit per sentiment class.

The entire encoder is fine-tuned with a *low* learning rate so that pre-trained knowledge is preserved while the new classifier layer is learned from scratch.

Loss functions:

Two alternatives were explored:

- 1 **Categorical Cross-Entropy** (the HuggingFace default):

$$\mathcal{L}_{\text{CE}} = - \sum_{c=1}^3 y_c \log p_c,$$

where y_c is the one-hot label and p_c the soft-max probability.

2 **Focal Loss** to combat class imbalance [8]:

$$\mathcal{L}_{\text{focal}} = -\alpha (1 - p_t)^\gamma \log p_t, \quad \gamma=2,$$

where p_t is the probability assigned to the true class and α is an optional class-weighting factor. The multiplicative term $(1 - p_t)^\gamma$ down-weights easy/high-confidence examples and forces the network to focus on hard or minority-class samples.

The choice {Cross-Entropy, Focal} was treated as a hyper-parameter.

Optimiser: training employed **AdamW** with $\beta_1=0.9$, $\beta_2=0.999$ and decoupled weight decay [loshchilov2019decoupled]. Weight decay λ was itself tuned (range 0–0.1) and *excluded* from bias and LayerNorm parameters.

Learning-rate schedule: a *linear warm-up / linear decay* schedule was used:

$$\eta(t) = \begin{cases} \frac{t}{T_{\text{warm}}} \eta_{\text{max}}, & 0 \leq t < T_{\text{warm}}, \\ \eta_{\text{max}} \left(1 - \frac{t - T_{\text{warm}}}{T_{\text{total}} - T_{\text{warm}}} \right), & T_{\text{warm}} \leq t \leq T_{\text{total}}, \end{cases}$$

where T_{warm} corresponds to 10% of training steps. Warm-up stabilises early optimisation when the randomly initialised classifier layer could otherwise cause gradient spikes.

Handling class imbalance: the neutral class ($y=1$) comprised the fewest examples. Two complementary strategies were integrated into the hyper-parameter search:

- (a) **Class-weighted loss:** loss weights were set inverse-proportional to class frequencies, so that an error on a neutral review incurred a higher penalty.
- (b) **Balanced sampling:** a **WeightedRandomSampler** over-samples minority classes and under-samples majority classes, producing class-balanced mini-batches each epoch.

The search space allowed *none*, *weighted loss*, or *sampler* (but not both simultaneously), letting Optuna select the combination that maximised validation macro-F₁.

Hyper-parameter grid: key parameters optimised via Optuna 5-fold cross-validation are summarised in Table 5.1. The resulting best configuration is reported in Table 5.2 together with final evaluation metrics.

Parameter	Search range / options
Learning rate	$1 \times 10^{-5} - 5 \times 10^{-5}$ (log-uniform)
Batch size	{8, 16, 32}
Weight decay λ	0 – 0.1 (uniform)
Epochs	{2, 3, 4, 5, 6}
Warm-up ratio	{0, 0.1, 0.2}
Loss function	{Cross-Entropy, Focal ($\gamma=2$)}
Imbalance handling	{none, class weights, sampler}

Table 5.1: Hyper-parameter search space explored with Optuna.

5.6 Hyper-parameter Tuning

Fine-tuning large language models is notoriously sensitive to the choice of optimisation settings. To avoid a manual, trial-and-error process we adopted *Optuna*³, an automatic hyper-parameter optimisation (HPO) framework that uses Bayesian optimisation with tree-structured Parzen estimators. The objective function was the **macro F₁** computed under *stratified 5-fold cross-validation* on the training split:

- In every Optuna *trial* a candidate configuration is sampled from the search space (Table 5.1).
- The training set is partitioned into five folds with the same class proportions (negative, neutral, positive) as the full corpus. The model is trained on four folds and validated on the remaining fold; this is repeated so each fold acts as validation once.
- The mean macro F₁ across the five validations is returned to Optuna, which updates its surrogate model and proposes a better configuration in the next trial.

Cross-validation—rather than a single train/validation split—gives a robust estimate of out-of-sample performance and guards against the risk that an *imbalanced* validation fold skews the ranking of parameter sets.

Search space. The tuned parameters and their ranges have already been listed in Table 5.1. In total, $T = 20$ **trials** were executed; early-stopping inside each fold prevented superfluous epochs when the validation score failed to improve for two consecutive epochs.

Best configuration. Table 5.2 reports the top-ranked hyper-parameters selected by Optuna together with the rationale that emerged from the search.

Insights

³<https://optuna.org/>

Hyper-parameter	Best value (Optuna)
Learning rate	2.98×10^{-5}
Batch size	16
Weight decay	0.072
Epochs	5
Warm-up ratio	0.1 (10% of steps)
Loss function	Focal Loss ($\gamma = 2$)
Class balancing	Sampler

Table 5.2: Optimal hyper-parameters discovered by Optuna (macro F_1 averaged over 5 folds: 0.832).

- *Focal loss* delivered a small but consistent gain over vanilla cross-entropy, rendering additional class weights or sampling unnecessary; it implicitly down-weighted the abundant positive class and allowed the model to focus on the rarer neutral reviews.
- A *moderate* learning rate ($\approx 3 \times 10^{-5}$), five epochs, and a 10 % warm-up phase produced the best bias-variance trade-off: lower rates converged too slowly, whereas higher rates or more epochs risked over-fitting.
- Batch size 16 proved to be the sweet spot on our hardware, offering a good gradient estimate while fitting comfortably in GPU memory for ruBERT-large.

These findings are in line with best-practice recommendations for transformer fine-tuning and provided a solid foundation for training the *final* model evaluated in the next section.

enumitem

5.7 Final Model Training and Evaluation

With the optimal hyper-parameters in Table 5.2, the classifier was re-trained on *all* training folds (i.e. the entire training set) and then evaluated on the previously unseen **hold-out test set**. Training lasted for 5 epochs with a peak learning rate $\eta = 2,98 \times 10^{-5}$, a 10% warm-up phase and *Focal Loss* ($\gamma = 2$) as the objective.⁴

Evaluation metrics. We report overall *Accuracy*, macro-averaged⁵ F_1 -score, and macro *ROC-AUC* (one-vs-rest, then averaged).⁶

⁴No additional class weighting or over-/under-sampling was applied; Focal Loss alone proved sufficient to mitigate class imbalance.

⁵The mean of per-class metrics, giving each class equal weight.

⁶Macro ROC-AUC reflects the model’s ranking ability and is insensitive to class imbalance or threshold choice.

Metric	Score (test set)
Accuracy	0.9410
Macro F_1	0.8480
Macro ROC-AUC	0.9547

Table 5.3: Performance of the final ruBERT-large model on the held-out test set.

Confusion matrix. Table 5.4 details the distribution of predictions versus ground truth. The model is highly reliable on clearly negative or positive reviews and attains respectable performance on the minority *neutral* class.

Actual \ Predicted	Negative	Neutral	Positive
Negative	372	19	58
Neutral	38	170	54
Positive	34	10	2855

Table 5.4: Confusion matrix for the three sentiment classes on the test set.

Interpretation. The overall accuracy of **94 %** and macro F_1 of **0.84** show that the model generalises well across all three sentiment categories. The ROC-AUC of **0.95** indicates excellent ranking capability: the classifier can separate each class from the others with high confidence.

In summary, the BERT-based pipeline shows strong performance in classifying Russian real estate reviews by sentiment. It performs especially well on clearly negative or clearly positive feedback, which is reflected in high accuracy and F_1 for those classes. The careful handling of class imbalance (through focal loss and tuning) helped the model not to overlook neutral reviews, though there is still room to improve the precision on the neutral class. The errors in the confusion matrix reveal that when the model does make mistakes, it tends to predict an adjacent sentiment (e.g., a borderline neutral review might be labeled positive or negative). This suggests that the model could further benefit from more training examples of neutral sentiment to sharpen that decision boundary. Overall, the final evaluation indicates the model is effective for sentiment analysis of Russian texts in this domain, successfully leveraging BERT’s powerful language understanding to classify customer opinions about developers.

6 Clustering Analysis

6.1 Data Integration and Feature Engineering

The raw information arrived in *two* independent, parser-generated datasets:

- a) **Review corpus** – a CSV with the developer name, the full review text and the per-review star `rating`;
- b) **Developer registry** – a CSV with structured attributes for each company: *year of foundation*, *number of houses built*, *average price per m^2* and the *estate class(es)* it builds.

We joined them on a normalised developer key (`dev_norm`)⁷, resulting in a single frame of $16\,741 \text{ rows} \times 18 \text{ columns}$. During the join we

- manually filled $\approx 2.3\%$ missing cells (eight companies) using official web sources;
- one-hot encoded the categorical field `estate_class_list` into $\{\text{econ}, \text{comfort}, \text{business}, \text{premium}\}$;
- computed `years_on_market` as $\text{year}_{\text{now}} - \text{year}_{\text{foundation}}$.

Final metadata vector. For every review we therefore have four *normalised* numeric features $\mathbf{y} = (y_1, y_2, y_3, y_4)$:

$$y_1 = \text{years_on_market}, \quad y_2 = \text{avg_sqm_price}, \quad y_3 = \text{cnt_houses_built}, \quad y_4 = \text{dev_norm},$$

and four binary one-hot components $\mathbf{c} = (c_{\text{econ}}, c_{\text{comfort}}, c_{\text{business}}, c_{\text{premium}})$. The composite feature vector is

$$\mathbf{x} = \left[\mathbf{e}_{\text{BERT}}, 0.30 \mathbf{y}, 0.10 \mathbf{c} \right] \in \mathbb{R}^{768+4+4}, \quad (1)$$

where $\mathbf{e}_{\text{BERT}} \in \mathbb{R}^{768}$ is the Sentence-BERT embedding of the lemmatised review text. The scale factors (0.30, 0.10) were chosen empirically so that non-text dimensions are roughly one order of magnitude smaller than the text embedding.

Before clustering we reduced the 776-D vectors to 100-D with UMAP (`n_neighbors=15`, `metric=cosine`) and applied ℓ_2 -normalisation (so Euclidean distance equals cosine distance)

[11].

⁷See `join_of_main_files.ipynb`; manual harmonisation was required because spelling of developer names differed between sources.

6.2 K-Means Clustering: Methodology and Rationale

K-Means [9] partitions the data into K clusters by minimising the *within-cluster sum of squares*:

$$\text{WCSS} = \sum_{i=1}^K \sum_{\mathbf{x}_j \in C_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|_2^2, \quad (2)$$

where C_i is cluster i and $\boldsymbol{\mu}_i = \frac{1}{|C_i|} \sum_{\mathbf{x}_j \in C_i} \mathbf{x}_j$. Algorithm 1 sketches Lloyd’s procedure. With `k-means++` initialisation and `n_init=auto` the solution stabilised at $K = 6$. The value of K was selected via *silhouette* analysis ($\max_{K=6} s_{\text{avg}} \approx 0.279$) and the inertia–elbow (additional WCSS drop $< 5\%$ beyond $K = 6$).

Algorithm 1 Lloyd’s K-Means (high-level)

- 1: Initialise centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$ (`k-means++`).
 - 2: **repeat**
 - 3: **Assignment:** $C_i \leftarrow \{\mathbf{x}_j : i = \arg \min_k \|\mathbf{x}_j - \boldsymbol{\mu}_k\|_2\}$.
 - 4: **Update:** $\boldsymbol{\mu}_i \leftarrow \frac{1}{|C_i|} \sum_{\mathbf{x}_j \in C_i} \mathbf{x}_j$.
 - 5: **until** assignments stop changing or WCSS improvement $< \varepsilon$
-

K-Means presumes roughly convex, equally dense clusters; this is acceptable because UMAP already projects the space into well-separated blobs (cf. Fig. ??). With a single hyper-parameter K the outcome is reproducible (clusters 0–5 stayed identical across ten random seeds) and each cluster (1 382–4 779 reviews) is large enough to serve as an interpretable *topic*.

6.3 HDBSCAN Clustering: Methodology and Comparison

Algorithmic background. HDBSCAN⁸ extends DBSCAN into a hierarchical framework that *simultaneously* (i) copes with variable-density clusters and (ii) marks low-density points as *noise*. Two hyper-parameters control the granularity:

- `min_samples` (k) – influences the local density estimate (analogous to the k in k -NN);
- `min_cluster_size` – the minimum number of points that may form a cluster.

For a point p let $\text{core}_k(p)$ be the distance to its k -th nearest neighbour. The *mutual-reachability distance* between two points is then

$$d_k^{\text{mreach}}(p, q) = \max\{\text{core}_k(p), \text{core}_k(q), \|p - q\|_2\}. \quad (3)$$

⁸Hierarchical Density–Based Spatial Clustering of Applications with Noise. See [2] and the project documentation [10].

Algorithm 2 Outline of HDBSCAN

- 1: Build the complete graph on all points using d_k^{mreach} as edge weights.
 - 2: Compute its minimum-spanning tree (MST).
 - 3: **Condense** the MST by sweeping a density threshold $\lambda \propto 1/d$ downward and recording cluster splits/merges $\Rightarrow$ a *cluster hierarchy*.
 - 4: Compute the *stability* $S(C) = \sum_{p \in C} (\lambda_{\text{death}}(p) - \lambda_{\text{birth}}(p))$ for each branch C .
 - 5: Select the set of clusters that maximises the total stability (points not selected become noise).
-

Parameter tuning and empirical outcome. We performed a Bayesian search with `Optuna` maximising the *mean silhouette* on non-noise points. The best configuration (`min_cluster_size = 15`, `min_samples = 9`) yielded

# clusters	# noise pts	% noise	$\bar{s}$ (non-noise)
113	11 143	66.5 %	0.342

Thus two-thirds of the 16 741 reviews were labelled *noise*, and the remaining reviews were split into many micro-clusters (often < 10 points).

Qualitative comparison with K-Means.

1. **Number of clusters.** HDBSCAN is non-parametric (no predefined K) and returned 113 clusters; K-Means, with $K = 6$, produced six broader, interpretable topics.
2. **Cluster shape.** HDBSCAN can capture arbitrarily shaped, variable-density groups—useful for moon-shaped manifolds—whereas K-Means presumes roughly convex blobs. After UMAP our vectors formed nearly spherical “islands”, so K-Means’ assumption was adequate.
3. **Noise handling.** The ability of HDBSCAN to label noise is advantageous when true outliers exist; here, however, its sensitivity caused over-fragmentation and excessive noise (66 %). K-Means assigns every point, yielding complete coverage required for downstream topic modelling.
4. **Stability.** Small adjustments of `min_cluster_size` or `min_samples` radically changed HDBSCAN’s output. K-Means, once K is fixed, is reproducible (`n_init=auto`, seeds varied).

Decision. Because HDBSCAN split semantically coherent themes into dozens of tiny fragments and discarded most reviews as noise, we elected to use **K-Means with $K = 6$** for the remainder of the study. K-Means strikes a better balance between *interpretability* (six topics easy to label) and *data fidelity* (every review is represented), while still attaining a respectable silhouette ($\bar{s}_{K=6} = 0.279$).

7 Topic Extraction with TF–IDF and BERT Embeddings

After assigning every review to one of the six clusters (Section 6), we derived *interpretable topics* for each cluster by combining contextual BERT embeddings with a **class-based TF–IDF** (*c-TF–IDF*) weighting [6]. In essence, each cluster is treated as a single “super-document”, and TF–IDF is re-computed at the *cluster* level.

7.1 Methodology: c-TF–IDF for Cluster Keywords

Class aggregation. Let V be the vocabulary and C_i the concatenation of all lemmatised reviews in cluster i . For any term $t \in V$ we define

$$\text{TF}(t, C_i) = \frac{f(t, C_i)}{\sum_{t' \in V} f(t', C_i)}, \quad (4)$$

$$\text{IDF}_c(t) = \log \frac{N_{\text{docs}}}{f(t)}, \quad (5)$$

$$\text{cTF–IDF}(t, i) = \text{TF}(t, C_i) \text{IDF}_c(t), \quad (6)$$

where $f(t, C_i)$ is the raw frequency of t in C_i and $f(t) = \sum_i f(t, C_i)$ its total corpus frequency. Equation (5) penalises globally frequent tokens, even if they appear in few clusters, thereby sharpening topic contrast.

Implementation workflow.

- 1 Lemmatise text and remove Russian stopwords.
- 2 Concatenate reviews per cluster to form C_i .
- 3 Compute $\text{cTF–IDF}(t, i)$ via Eqs. (4)–(6).
- 4 For each cluster sort terms by score and keep the top $n = 20$ keywords.

7.2 Topics obtained

Table 7.1 lists the ten highest-scoring keywords for each cluster; these succinctly label the six latent themes that shape customer perception of developers.

Table 7.1: Top-10 c-TF-IDF keywords per cluster.

Cluster	Keywords (ranked)
0: <i>Transport</i>	метро, минута, доступность, парковка, транспорт, станция, пешком, район, удобный, дорога
1: <i>Layout / Value</i>	планировка, отделка, дом, цена, комплекс, вариант, хороший, купить, квартира, качество
2: <i>Family & Infrastructure</i>	школа, детский, сад, площадка, парк, семья, рядом, инфраструктура, территория, дети
3: <i>Delays & Handover</i>	ключ, срок, ждать, застройщик, месяц, сдача, получить, менеджер, год, документ
4: <i>Moscow vs Suburb</i>	Москва, район, московский, центр, прописка, МКАД, город, новый, близко, плюс
5: <i>Quality Complaints</i>	шумоизоляция, ужасный, плохой, проблема, сосед, УК, стена, лифт, работать, парковка

Interpretation. The discriminative tokens validate the semantic coherence of the clusters (e.g. Cluster 5 clearly represents *Noise & Quality Issues*), while remaining fully transparent to human readers. These six topics serve as explanatory features for the trust metric introduced in Section 8.

8 S-Index Calculation (Developer Trust Metric)

In this chapter we introduce and evaluate the **S-index**, a custom metric we devised to quantify developer trustworthiness based on review content and metadata. The S-index is essentially a score computed for *each* developer. It is designed to reflect how positive or negative the overall sentiment is towards that developer while *explicitly* accounting for **which topics** the sentiment addresses. A higher S-index indicates higher trust or satisfaction with the developer, whereas a lower (or negative) S-index indicates distrust or prevalent negative feedback.

8.1 3.1 Defining the S-Index and Its Relevance

Formal definition. Intuitively, not all mentions are equal – complaints about certain topics (e.g. build quality) are more damaging than others (e.g. location). Hence we weight sentiment per topic. Let

- N_d – total number of reviews for developer d ;
- $\text{pos}_{d,k}$ – number of *positive* reviews of d in topic cluster k ;
- $\text{neg}_{d,k}$ – number of *negative* reviews of d in the same cluster;
- w_k – weight expressing the importance of topic k .

Then

$$\boxed{\text{S-index}(d) = \sum_{k=0}^{K-1} w_k \left(\frac{\text{pos}_{d,k}}{N_d} - \frac{\text{neg}_{d,k}}{N_d} \right)} \quad (3-1)$$

For each topic k we take the difference between the proportion of positive and negative reviews and multiply it by w_k . Mostly positive reviews in an important topic push the S-index up; mostly negative reviews pull it down.

Why not just use the average star rating? Because a *single* star rating hides what people actually talk about. Two developers may both average 4 stars, yet one gets negative reviews on build quality (critical) while the other gets nit-picks about parking. Our S-index penalises the former more heavily, surfacing meaningful differences the star average misses.

Normalisation. The raw score of (3-1) lies in $[-1, 1]$. For readability we later map it to $[0, 1]$ and finally to the familiar 0–5 “star-like” range (Section 8.4).

8.2 3.2 Approaches Tested for S-Index Computation

To obtain the weights w_k we evaluated two families of methods.

3.2.1 Approach 1 – Unsupervised Cluster-Sentiment Heuristic

We assumed each cluster carries an *intrinsic* polarity. Over the whole corpus we computed

$$v_k = P_k - N_k, \quad P_k = \frac{\text{all positive reviews in } k}{\text{all reviews}}, \quad N_k = \frac{\text{all negative reviews in } k}{\text{all reviews}},$$

so $v_k \in [-1, 1]$. We then linearly rescaled to $[0, 1]$:

$$w_k^{(H)} = \frac{v_k - \min_j v_j}{\max_j v_j - \min_j v_j}.$$

Clusters dominated by complaints (Cluster 5) received weights near 0, those with mostly praise (Cluster 2) near 1. Plugging $w_k^{(H)}$ into (3-1) produced the *KM-heuristic S-index*. Although simple, this method ignores how topic sentiment interacts with individual developers; we therefore turned to supervised learning.

3.2.2 Approach 2 – Supervised Models

We experimented with **two** supervised variants that use the same feature vector

$$\mathbf{x}_d = [p_{d,0} - n_{d,0}, \dots, p_{d,5} - n_{d,5}] \in [-1, 1]^6,$$

where $p_{d,k}$ and $n_{d,k}$ are proportions of positive/negative reviews of developer d in cluster k .

(a) Ordinary Least-Squares Regression (baseline, rejected).

- 1 **Target.** Continuous average rating $\bar{r}_d \in [1, 5]$.
- 2 **Model.** $\hat{r}_d = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_d$.
- 3 **Validation.** LOOCV on 38 developers. Table 8.1 shows:

Table 8.1: LOOCV metrics for linear regression.

	R^2	MAE	RMSE
Mean of 38 folds	0.242	0.723	1.102

Additional diagnostics: Spearman $\rho = 0.873$; residuals systematically under-predict low-rated developers; bootstrap sign flips for β_3, β_5 in ~ 15 *Conclusion:* accuracy too poor, coefficients unstable — model discarded.

(b) Logistic Regression (final choice).

- 1 **Target.** Binary label: high-trust ($\bar{r}_d \geq 4$) =1, low-trust =0.
- 2 **Model.** $\Pr(y_d = 1) = \sigma(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_d)$, solver `liblinear`, mild L_2 .
- 3 **Validation.** LOOCV: Accuracy 0.895, Macro-F1 0.879, AUC 0.944, Spearman $\rho = 0.897$, Gini 0.659.
- 4 **Weights.** $w_k^{(\text{LR})} = \beta_k$ (normalised).

Signs are intuitive: $w_5 = -1.47$ (complaints, highly negative), $w_3 = -0.68$ (delays, negative), $w_2 = +0.54$ (family infrastructure, positive), $w_{0,4} \approx +0.2$ (location, mild), $w_1 \approx +0.1$ (layout/value, small positive).

8.3 3.3 Evaluation of All S-Index Variants

Table 8.2: Quality comparison of weight-learning strategies.

Variant	Spearman ρ	Accuracy	Macro-F1	Gini
Heuristic (KM)	0.852	0.816	0.792	0.612
Linear (OLS)	0.873	0.842	0.821	0.631
Logistic (chosen)	0.897	0.895	0.879	0.659

The logistic formulation clearly dominates, so we adopt $\text{S-index}_{\text{LR}}$.

8.4 3.4 Rescaling to 0–5 and Top-10 Leader Board

We map the logistic score to a 0–5 trust gauge:

$$\text{S-index}_{0-5} = 5 \times \text{S-index}_{\text{LR}}.$$

Table 8.3: Top-10 developers by S-index_{0-5} .

Developer	S-index 0–5	Avg. rating
October Group	5.00	4.91
FORMA	4.14	4.84
Element	4.00	4.83
Галс-Девелопмент	4.08	4.72
Строительный холдинг «Сенатор»	3.85	4.71
ДОНСТРОЙ	4.00	4.70
Группа «Аквилон»	3.62	4.64
Брусника	3.66	4.64
RBI	3.97	4.63
Optima Development	3.79	4.58

A scatter-plot of S-index vs. raw rating (Fig. 3.1) shows a near-monotonic trend ($\rho = 0.897$) but with outliers where topic weighting penalises or rewards beyond the star average.

8.5 3.5 Pre-processing Pipeline for the Logistic Model

1 **Sentiment labelling:** 0 = neg, 1 = neutral, 2 = pos. Neutral reviews excluded from $p_{d,k}, n_{d,k}$.

- 2 **Aggregation:** counts of pos/neg per cluster, total reviews N_d .
- 3 **Normalisation:** convert counts to proportions, build $\mathbf{x}_d$.
- 4 **Training:** 70/30 split + LOOCV, solver `liblinear`, mild L_2 .
- 5 **Metrics:** Accuracy, Precision, Recall, F1, macro-F1, Spearman ρ , Gini inequality.

8.6 3.6 Summary

The S-index fuses NLP (topic + sentiment) with supervised learning. Logistic weighting outperforms both a naive heuristic and a linear model, achieving $\rho = 0.897$ with star ratings while exposing: *quality complaints* and *delays* damage trust most, *family infrastructure* uplifts it, and *location* adds a mild bonus. This nuanced, actionable metric surpasses the coarse star average and can guide both consumers and developers.

9 Conclusion

Over the course of this study we developed an **end-to-end, topic-aware «Social» (S-) index** that measures trust in Russian housing developers based on 18 048 user reviews. The pipeline marries state-of-the-art Natural Language Processing with classical machine learning; every step is fully reproducible in the accompanying notebooks.

Key achievements

1. **High-quality data foundation.** A Playwright crawler extracted texts and meta-data from Avito and CIAN, followed by rigorous cleaning and lemmatisation (*natasha*)—yielding the first consolidated corpus for 38 major developers.
2. **Accurate sentiment labelling.** A fine-tuned *ruBERT-large* classifier [4] reached **94 % accuracy** and **0.848 macro-F1** on a held-out set thanks to focal loss [8] and Optuna hyperparameter optimisation [1].
3. **Robust thematic structure.** Sentence-BERT embeddings, four numeric developer features and one-hot estate-class vectors were concatenated, reduced with UMAP [11] and clustered via $K = 6$ **K-Means** (silhouette 0.279). A comparison with HDBSCAN [2] showed 66 % noise and 113 micro-clusters, justifying the K-Means choice.
4. **Human-readable topics.** Class-based TF-IDF (c-TF-IDF) from BERTopic [6] produced coherent keyword sets: e.g. *шумоизоляция*, *ужасный*, *сосед* for “Quality complaints” and *школа*, *детский*, *площадка* for “Family & infrastructure”.

5. Trust metric. Three weighting schemes were evaluated:

- *Unsupervised heuristic* (Spearman $\rho = 0.852$);
- *Linear regression* baseline ($R^2 = 0.242$, MAE 0.723, RMSE 1.102, $\rho = 0.873$);
- *Logistic regression*—**chosen** (accuracy 0.895, macro-F1 0.879, AUC 0.944, $\rho = 0.897$, Gini 0.659)[13].

Learned weights were intuitive: $w_5 = -1.47$ (build-quality complaints, most damaging), $w_2 = +0.54$ (family infrastructure, most beneficial). Scores were linearly rescaled to **0–5**.

Insights

- **Topic matters.** Developers with identical 4-star averages can differ by >1.5 points on the S-index once topic weighting is applied.
- **Actionability.** Cluster-specific sentiment pinpoints weaknesses: frequent Cluster 5 complaints depress the score more than neutral Cluster 0 location remarks.
- **Scalable ESG monitoring.** The pipeline ingests thousands of reviews in minutes, enabling near-real-time surveillance of trust dynamics.

Limitations

- Data sources exclude Telegram, VK and offline surveys.
- Basic spam heuristics flag bots; advanced detection is future work.
- Neutral class is under-represented (7 %), blurring class boundaries.
- Logistic weights are global; time-varying or developer-specific weights could capture shocks more precisely.

Future work

1. Extending the dataset to other platforms
2. Incorporating additional metadata (e.g., court cases)
3. Dynamic monitoring of reputation changes over time
4. Introduce rolling-window training to trace reputation over time.
5. Apply graph analytics to link reviewers, projects and locations.
6. Add SHAP explanations and attention visualisation for sentence- level justifications.

The resulting S-index therefore represents a *nuanced, actionable and fully transparent* trust metric, improving upon coarse star averages and offering both strategic insights for developers and

an intuitive gauge for home-buyers.

References

- [1] Takuya Akiba, Shotaro Sano, Takeru Yanase, Tatsuya Ohta, and Masanori Koyama. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: (2020).
- [2] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. “Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection”. In: *ACM TKDD* (2015).
- [3] Yu-Min Tseng Chung-Chi Chen. “Multi-lingual esg issue identification”. In: *Proceedings of the Fifth Workshop on Financial Technology and Natural Language Processing (FinNLP)*. 2023.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv preprint arXiv:1810.04805* (2018).
- [5] github. *The Illustrated BERT, ELMo, and co*. URL: <https://jalammar.github.io/illustrated-bert/> (visited on Jan. 1, 2023).
- [6] Maarten Grootendorst. “BERTopic: Neural topic modelling with class-based TF-IDF”. In: *arXiv preprint arXiv:2203.05794* (2022).
- [7] IBM. *Stemming and Lemmatization*. 2011.
- [8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. “Focal Loss for Dense Object Detection”. In: (2017).
- [9] Stuart P. Lloyd. “Least Squares Quantization in PCM”. In: *IEEE Transactions on Information Theory* 28.2 (1982), pp. 129–137. DOI: [10.1109/TIT.1982.1056489](https://doi.org/10.1109/TIT.1982.1056489).
- [10] Leland McInnes, John Healy, and Stuart Astels. *HDBSCAN: Hierarchical Density Based Clustering*. <https://hdbscan.readthedocs.io/>. Software documentation, accessed 27 May 2025. 2025.
- [11] Leland McInnes, John Healy, and James Melville. “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction”. In: *arXiv:1802.03426*. 2018.
- [12] Medium.com. *Sentiment Analysis with BERT using Huggingface*. URL: https://medium.com/@manjindersingh_10145/sentiment-analysis-with-bert-using-huggingface-88e99deec9a (visited on Nov. 19, 2023).
- [13] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. “Scikit-learn: Machine Learning in Python”. In: *JMLR* 12 (2011), pp. 2825–2830.

- [14] Elvys Linhares Pontes, Mohamed Benjannet, and Lam Kim Ming. “Leveraging BERT Language Models for Multi-Lingual ESG Issue Identification”. In: *arXiv:2309.02189v1* (2023).
- [15] JSC SBER. *ESGify: open-source NLP-model for classifying ESG-risks*. URL: <http://www-cs-faculty.stanford.edu/~uno/abcde.html> (visited on Jan. 15, 2025).
- [16] Robert Louka Srishti Mehra and Yixun Zhang. “Esgbert: Language model to help with classification tasks related to companies environmental, social, and governance practices.” In: *arXiv preprint arXiv:2203.16788* (2022).